TruckLab Kinematic Model: Simulation and Validation

Tractor–Trailer single-track kinematic model implementation and empirical validation using OptiTrack data

Authors: Jakub Kasprzyk, Koen Scholten: k.j.scholten@student.tue.nl

Introduction

In this assignment you will implement, simulate, and experimentally validate a single-track kinematic model of a Tractor–Trailer configuration. The work proceeds from first principles—writing the equations of motion and building a Simulink diagram—to quantitative analysis and comparison against OptiTrack data collected in the TruckLab.

By completing all exercises, you will gain knowledge and skills in following topics:

- Implementing the model in Simulink (block-diagram realization of $\dot{x}, \dot{y}, \dot{\psi}_1, \dot{\psi}_2$) and running discrete-time simulations.
- Fitting geometric primitives (circles) to (x, y) trajectories via Least Squares and interpreting the results (turning radius, center).
- Understanding the relation between the steering angle δ and turning radius of the tractor and trailer.
- Verifying geometric properties (heading–radius perpendicularity) with both slope and dot-product tests, to sanity check your single-track kinematic model.
- Running the developed Simulink models in the actual physical 'TruckLab' set up, and learning more about how it works.
- Processing noisy experimental data: smoothing position and heading signals, differentiating them numerically, and transforming velocities between the global and body frames.
- Characterizing the vehicle's equivalent wheelbase and understeer gradient from steady-state speed sweeps, and relating this back to the kinematic model.

Upon completion, you should be able to derive and implement a tractor–trailer kinematic model, run it in Simulink with meaningful numerical settings, and evaluate it quantitatively against experimental data acquired in the TruckLab.

Submission: The deliverable for this assignment is a *single PDF report per group* covering Exercises 3.2 through 3.7. Exercises 1 and 2 are preparatory and are not handed in, though their implementation is assumed throughout.

- **Scope.** Address every exercise from 3.2 to 3.7, in order, under a clearly labelled heading.
- **Conciseness.** Keep answers compact and to the point: state only what is needed to interpret the result, without padding or repeating information.
- **Figures.** Export all plots in a *vector* format (PDF or EPS), so they stay sharp at any zoom level.
- **Interpretation.** Present no result without explanation. For each figure, table, and fitted number, state what it shows, whether it matches what the kinematic model predicts, where it does not, and which physical effect accounts for the difference.
- **Numerical values.** Report fitted parameters (e.g. a, b, L_{eq}) with units and a sensible number of significant digits, and state the filter settings (`w_pos_vel`, `w_yaw_rate`, `t_start`, `t_end`) used to obtain them.

Throughout the Exercise 1 and 2 of this assignment we will be developing, implementing and reasoning with the outputs of the single-track kinematic model of a Tractor-Trailer configuration.

Intermezzo 1: Vehicle Single-track Kinematic Model

This is a kinematic model of a tractor-trailer combination taken from Gosar et al. (2025):

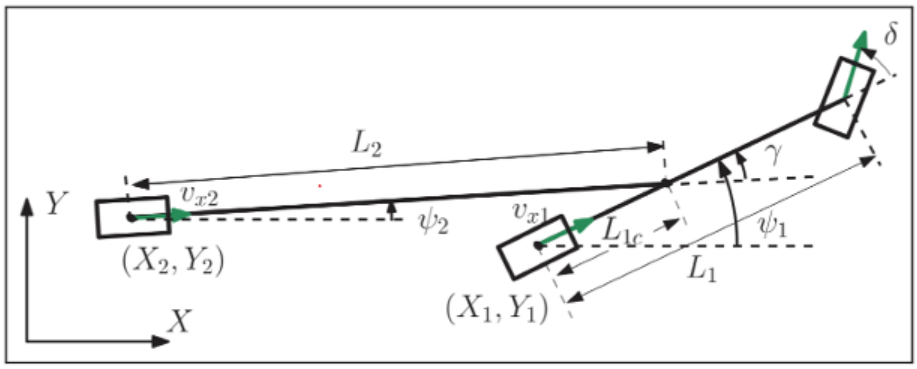

Figure 1: Kinematic model.

The model portrayed in Figure 1 is governed by the following equations of motion for the tractor:

$$\dot{x} = v_{x_1} \cos(\psi_1), \tag{1}$$

$$\dot{y} = v_{x_1} \sin(\psi_1), \tag{2}$$

$$\dot{\psi}_1 = \frac{v_{x1}}{L_1} \tan(\delta), \tag{3}$$

and by the following equation of motion describing the heading of the trailer:

$$\dot{\psi}_2 = \frac{v_{x_1}}{L_2} \left[\sin\gamma + \frac{L_{1c}}{L_1} \tan\delta \cos\gamma \right], \tag{4}$$

where γ denotes the articulation angle and is defined as $\gamma = \psi_1 - \psi_2$.

Exercise 1: Tractor kinematic model formulation and implementation

In the first part of the assignment, we will focus on the generic single-track kinematic model adapted to the geometry of the RC tractor model. For that purpose, we will be using the Matlab script and Simulink model templates:

- 'run_Tractor_SingleTrack_Model.m', and
- 'Tractor_SingleTrack_Simulink.slx'.

Both files are included in the provided zip folder.

Exercise 1.1: Tractor single-track Simulink model implementation

[Tractor_SingleTrack_Simulink.slx]

Open 'Tractor_SingleTrack_Simulink.slx' and inside the Subsystem called Tractor Single-track Model implement equation (1) to (3) in Simulink block diagram form.

At this point, you can run the cell titled Tractor animation by pressing Ctrl + Enter. If the equations of motion have been implemented correctly, you should see a step-wise animation of the tractor moving along a circular path.

Next, we want to quantify the tractor's turning radius. Simulink provides discretized rear axle position data (x, y), which we can use to fit a circle. From this fit, we obtain both the circle's center coordinates and, most importantly, its radius.

Intermezzo 2: Fitting a circle over x, y data

The equation of a circle is given by

$$(x-a)^2 + (y-b)^2 = r^2,$$

which expands to

$$x^2 + y^2 - 2ax - 2by + a^2 + b^2 - r^2 = 0.$$

Rearranging, we obtain the general quadratic form

$$x^2 + y^2 + Dx + Ey + F = 0,$$

where

$$D = -2a, \quad E = -2b, \quad F = a^2 + b^2 - r^2.$$

Given measurement data points (x_i, y_i), the circle parameters D, E, F can be obtained by solving the linear system $Ax = b$

$$\underbrace{\begin{bmatrix} x_1 & y_1 & 1 \\ x_2 & y_2 & 1 \\ \vdots & \vdots & \vdots \\ x_n & y_n & 1 \end{bmatrix}}_{A} \underbrace{\begin{bmatrix} D \\ E \\ F \end{bmatrix}}_{x} = - \underbrace{\begin{bmatrix} x_1^2 + y_1^2 \\ x_2^2 + y_2^2 \\ \vdots \\ x_n^2 + y_n^2 \end{bmatrix}}_{b}.$$

If the points (x_i, y_i) lay exactly on a circle, this system would have an exact solution. However, due to numerical errors or noisy measurement data, the system is typically *overdetermined* (more equations than unknowns). In this case, we cannot solve $Ax = b$ exactly.

Instead, we solve the least-squares problem

$$\min_x \ \|Ax - b\|_2^2,$$

which finds the parameters D, E, F that minimize the squared algebraic error across all data points.
Once D, E, F are obtained, the circle parameters are recovered as

$$a = -\frac{D}{2}, \qquad b = -\frac{E}{2}, \qquad r = \sqrt{\left(\frac{D}{2}\right)^2 + \left(\frac{E}{2}\right)^2 - F}.$$

Exercise 1.2: Circle fit implementation

[`\+helpers\fit_circle.m`]

In the folder `\+helpers`, locate the Matlab function script `fit_circle.m`. Within the designated section of this script, implement the least-squares optimization procedure for fitting a circle to the data, as outlined in Intermezzo 2.

During any kind of development process, a good practice is to perform sanity checks throughout its duration. Those are quick checks that most likely will not makes its way to the final report but give us a chance to reason with the results of our model/simulation.

In our case, we will be performing two sanity checks:

- comparing the average turning radius throughout the simulation with the radius of the fitted circle
- verifying if the heading of the vehicle is perpendicular to its turning radius, as this is an important characteristic of our kinematic model.

Exercise 1.3: Average radius implementation

[`\+helpers\compute_radius.m`]

In the folder `\+helpers`, locate the Matlab function script `compute_radius.m`. Within the designated section of this script, compute an average Euclidean distance between center of the fitted circle and the position of the vehicle throughout the simulation.

Intermezzo 3: Checking perpendicularity of two lines

A common way to test whether two lines are perpendicular is through their slopes. If the slope of the first line is m_1 and the slope of the second line is m_2, then the lines are perpendicular if

$$m_1 m_2 = -1.$$

In our context, the heading of the vehicle defines the first line with slope

$$m_{\text{heading}} = \tan(\psi_1),$$

while the line from the circle center (a, b) to the rear axle position (x, y) defines the turning radius with slope

$$m_{\text{radius}} = \frac{y - b}{x - a}.$$

By multiplying these slopes, one can verify whether the heading and the radius are perpendicular. However, this slope-based test can be numerically unstable (e.g. when the line is vertical, its slope becomes undefined) and prone to floating-point error. A more robust way is to use vector algebra. Each line can be represented by a direction vector:

$$\mathbf{v} = \begin{bmatrix} \cos\psi_1 \\ \sin\psi_1 \end{bmatrix}, \qquad \mathbf{u} = \begin{bmatrix} x - a \\ y - b \end{bmatrix}.$$

The lines are perpendicular if their dot product vanishes:

$$\mathbf{v} \cdot \mathbf{u} = 0.$$

In practice, due to numerical error, one declares perpendicularity if

$$|\mathbf{v} \cdot \mathbf{u}| < \varepsilon,$$

with ε a small tolerance (e.g. 10^{-6}). This vector formulation is both elegant and robust, and avoids the singularities that occur in the slope-based approach.

Exercise 1.4: Slope-based perpendicularity check

[`\+helpers\compute_slope.m`]

In the `compute_slope.m` Matlab function script implement the slope-based perpendicularity check detailed at the beginning of the Intermezzo 3.

Exercise 1.5: Dot-product-based perpendicularity check

[`\+helpers\check_perpendicularity.m`]

In the `check_perpendicularity.m` Matlab function script implement the dot-product-based perpendicularity check detailed in the later part of the Intermezzo 3.

Exercise 1.6: Tractor heading angle analysis

Plot the heading angle of the tractor over simulation time in [`run_Tractor_SingleTrack_Model.m`]. As you can see the angle is accumulating throughout the duration of the simulation. Numerically it is not an issue, even when later computing the articulation angle γ, but given our coordinate and angle sign convention it is counter intuitive. In your implementation of the `psi1_dot` equation please wrap the heading angle, so $\psi_1 \in (0, 2\pi)$ - [`Tractor_SingleTrack_Simulink.slx`]. (Hint: Use the `Simulink\Math Operations\Mod` for that task)

Exercise 2: Tractor–Trailer kinematic model implementation

In this part of the assignment, we build on the work already completed for the tractor single–track (bicycle) model. The aim is to extend this formulation with (4), resulting in the kinematic model of a tractor–trailer configuration.

For this exercise, you will use the following Matlab script and Simulink model templates:

- `run_TractorTrailer_SingleTrack_Model.m`
- `Tractor_Trailer_SingleTrack_Simulink.slx`

Both files are provided in the accompanying zip folder. The developed model will later be validated against the physical, scaled RC tractor–trailer setup in the Automotive Technology Lab.

Exercise 2.1: Simulink model expansion: Trailer heading angle ψ_1

Open `Tractor_Trailer_SingleTrack_Simulink.slx` and navigate to the Subsystem `Tractor Trailer Single-Track Model`. Copy the equations of motion from `Tractor_SingleTrack_Simulink.slx` and extend them by implementing equation (4) in block diagram form.

As in Exercise 1.6, remember to wrap the angle ψ_2 appropriately. To compute the trailer heading angle ψ_2, first determine the articulation angle using the unwrapped headings ψ_1 and ψ_2.

Exercise 2.2: Quantitative kinematic model analysis

[`run_TractorTrailer_SingleTrack_Model.m`]

In the designated, later part of the '`run_TractorTrailer_SingleTrack_Model.m`' Matlab script, create a 3-by-1 subplot capturing the heading angles ψ_1, ψ_2, and the articulation angle γ over the course of the simulation. What can you say about the articulation angle γ?

Exercise 3: TruckLab Hands-on and kinematic model validation

This part of the assignment will be carried out in person in the Automotive Technology Laboratory, located in the Gemini Nord building at TU/e. It is required to finish Exercises 1 and 2 beforehand. Otherwise, some of the scripts you will use for exercise 3 will not work.

Preparation: The TruckLab has 5 work PCs that can be used to run the experiments in the remaining exercises. Before starting the exercises on one of these PCs, perform the following steps to prepare it:

- Copy the `OptiTrack\MATLAB\Plugin\1.1.0` folder from `C:\Users\Public\Documents` to your local Documents folder and extract all contents from the zip file
- In MATLAB 2025b, open the Documents folder, right-click the `OptiTrack\MATLAB\Plugin\1.1.0` folder, and select *Add to Path → Selected Folders and Subfolders*
- Open `getRigidBodyPose.m` and set `clientIP` to the PC's IP address, which can be read from the sticker on the top of the PC.
- Test the setup by running `getRigidBodyPose(ID)`, where `ID` is the truck `ID` of a truck in the TruckLab. getRigidBodyPose.m is located in assignment folder.
- If a MATLAB window pops up that asks you to localize NatNetLib,dll,

Exercise 3.1: Test Ride of the Tractor-Trailer

[`Log_Optitrack_TruckLab_A1.slx`]

In this exercise, you will be driving around with the steering wheel and pedals. In the meantime, you will be logging the measurement data on one of the 5 work PC's. Before you start, ask the TA to set up the PC with the steering wheel and pedals.

The TruckLab has three tractors (1, 2, 3), each publishing and receiving commands on its own ROS 2 topics, and any PC can control any tractor. Before driving with the steering wheel and pedals, make sure the Simulink model is targeting the tractor you are actually using:

- Locate the `VehicleActuation` subsystem and open the ROS 2 `Publish` block inside it.
- Select the topic matching your tractor: `/tractor1/cmd_vel`, `/tractor2/cmd_vel`, or `/tractor3/cmd_vel`.
- Double-check the number on the physical tractor in front of you against the topic you selected — since any PC can control any tractor, picking the wrong topic means commanding someone else's vehicle.

Once the topic is set, **Drive around** the scaled distribution center in the TruckLab and familiarize yourself with the steering. At this stage, try applying a constant throttle and steering angle, causing the tractor-trailer to drive in circles.

Make sure that:

- the `Stop Time` is set to `inf`,
- that `Simulation Pacing` is enabled and set to `1`.

In the mean time on the work PC, run the logging script while driving around. The optitrack measurements will be saved in the workspace as a SimulationOutput file with the name `s`. Take

a good look inside the file to see what is being logged. The purpose of this exercise is to establish a connection with the TruckLab setup and ensure you can:

- manually drive the tractor around the TruckLab,
- log both tractor and trailer data streamed by OptiTrack,
- access this data in your Matlab Workspace after closing the Simulink model.

> **Tip:** While getting a feel for the steering in Exercise 3.1, it is worth identifying the smallest absolute steering angle δ for which the tractor–trailer can still safely and reliably complete a full circle. Keep this δ in mind, as it will be a useful reference point when choosing your steering angles in Exercise 3.3.

Exercise 3.2: Baseline Identification

[`Fixed_Velocity_Fixed_Delta_Trucklab_A1.slx`]

Using the provided Simulink model, drive the tractor–trailer combination in a circle and log the OptiTrack data using the following parameters:

- $\delta = 10°$
- $V = 0.1\,\mathrm{m\,s^{-1}}$

Drive at least one full circle and then save the measurement data. Then use `physical_vs_kinematic.m` to compare the measured trajectory against the theoretical trajectory predicted by the kinematic model developed in Exercises 1 and 2.

You will likely observe a discrepancy between the theoretical and measured trajectories. This discrepancy serves as the baseline for the rest of the assignment: in the following exercises, you will use system identification to characterize this mismatch and refine the model's parameters accordingly.

In your report, include the resulting trajectory figures and state your observations: do the tractor and trailer follow the same trajectory as predicted by the kinematic model? Describe and explain any discrepancies you observe.

Exercise 3.3: Velocity sweep experiment

[`Velocity_Sweep_Fixed_Delta_TruckLab_A1.slx`]

For this experiment, the steering angle δ is held *constant* for the full duration of a run, while the vehicle speed is swept continuously using a velocity ramp block. This produces a single run per steering angle that covers a continuous range of speeds, from which the speed-dependent behavior of the vehicle can be extracted directly. Configure the velocity input as a ramp increasing from a low speed to the highest speed you can safely and reliably drive in the TruckLab, and set a fixed steering angle δ for the duration of the run. Log the resulting data as in Exercise 3.1–3.2.

Repeat the full velocity sweep for at least four different steering angles δ. Save each run as a separate `.mat` file, named after its commanded steering angle (e.g. `7.5deg.mat`). You will load and process all of these files together in the next exercises. In your report, state the steering angles and speeds you used for each run, and give a short explanation of why you chose them.

Choosing your steering angles: Later in this assignment, you will approximate $\tan\delta \approx \delta$ (small-angle approximation) to simplify the kinematic model. This approximation is not exact, and its error grows with δ. Before choosing your set of steering angles, estimate analytically, not just from a plot, how the relative error of this approximation depends on δ, and up to what steering angle you consider the approximation acceptable.

Intermezzo 4: Expressing a Velocity Vector in the Body Frame

When processing OptiTrack data, position and velocity are naturally expressed in the *global* frame (X, Y), the fixed coordinate system of the lab. However, for vehicle dynamics we are typically interested in the *body-frame* longitudinal velocity: how fast the vehicle moves along its own centerline, i.e., what a speedometer inside the vehicle would read.
Let ψ be the heading angle of the vehicle, defined as the angle between the body-frame x-axis and the global X-axis. A vector expressed in the body frame, with components (v_x, v_y), relates to its global-frame components $(v_{x,g}, v_{y,g})$ through the standard 2D rotation matrix:

$$\begin{bmatrix} v_{x,g} \\ v_{y,g} \end{bmatrix} = \begin{bmatrix} \cos\psi & -\sin\psi \\ \sin\psi & \cos\psi \end{bmatrix} \begin{bmatrix} v_x \\ v_y \end{bmatrix}.$$

This matrix rotates a body-frame vector *into* the global frame. In our case, however, we have the opposite situation: we *measure* $(v_{x,g}, v_{y,g})$ directly (e.g. by differentiating the logged x, y position), and we want to recover the body-frame velocity (v_x, v_y).

Inverting the rotation. Since a rotation matrix is orthogonal, its inverse is simply its transpose:

$$\begin{bmatrix} v_x \\ v_y \end{bmatrix} = \begin{bmatrix} \cos\psi & \sin\psi \\ -\sin\psi & \cos\psi \end{bmatrix} \begin{bmatrix} v_{x,g} \\ v_{y,g} \end{bmatrix}.$$

Reading off the first row gives the body-frame longitudinal velocity in terms of the measured global velocity components:

$$v_x = v_{x,g}\cos\psi + v_{y,g}\sin\psi.$$

A note on the heading angle. Because this transformation uses $\cos\psi$ and $\sin\psi$ directly, ψ must be a continuous, *unwrapped* angle (see Exercise 1.6) — not one artificially wrapped into $(0, 2\pi)$. If a wrapped heading is used here, the differentiation step used to obtain the yaw rate ω_z (and any quantity derived from it) will contain jumps at the wrap points.

Exercise 3.4: Moving Average Filter + Differentiation

[`Velocity_sweep_analysis.m`]

Raw measurement data in general contains noise, and numerical differentiation (`gradient`) amplifies that noise. Before differentiating, smooth the position and heading signals using a moving-average filter (`movmean`), with separate window lengths for position/velocity and for heading/yaw rate.

In the provided analysis script, for each loaded run:

- filter the global position and heading data by applying a moving average using the `movmean` function
- differentiate them with respect to time to obtain the global velocity components

- project the global velocity onto the vehicle heading to obtain the body-frame longitudinal velocity v_x (see the intermezzo on this transformation).

The code to plot the velocity v_x and heading angle ψ is already provided at the end of the script. Use these plots to tune the parameters:

- `w_pos_vel`: window length for position and velocity [s]
- `w_yaw_rate`: window length for heading and yaw rate smoothing [s]
- `t_start`: start time to trim [s]
- `t_end`: end time to trim [s]

In your report, include the resulting plots and state the filter window lengths you used for each run. Also, comment on the effect of over-filtering: what happens to the signal if the window length is chosen too large?

Exercise 3.5: Yaw Velocity Gain

[`Velocity_sweep_analysis.m`]

In the main loop of the provided analysis script, calculate the yaw velocity gain ω_z/δ for each run, using only the valid samples identified earlier.

Using a small-angle approximation on δ in the kinematic model from Intermezzo 1, derive an expression for the *theoretical* yaw velocity gain in terms of the vehicle speed v_x and wheelbase L.

Outside the main loop, plot the measured yaw velocity gain against v_x for every run in a single figure, together with your theoretical expression evaluated over the same speed range. What do you observe? In particular, how does the measured yaw velocity gain behave relative to the theoretical one as speed increases, and what might explain this?

Exercise 3.6: Equivalent wheelbase

[`Velocity_sweep_analysis.m`]

Using the same valid samples from Exercise 3.5, compute the *equivalent wheelbase*

$$L_{\text{eq}} = \frac{v_x\,\delta}{\omega_z}, \tag{5}$$

together with v_x^2, for every run in the main loop of the provided analysis script. Plot L_{eq} against v_x^2 for all steering angles on a single figure outside of the main loop, and mark the measured wheelbase L as a horizontal reference line. In your report, discuss what L_{eq} represents physically and how it relates to the vehicle's actual measured wheelbase L.

Exercise 3.7: Characterization equivalent wheelbase

[`Velocity_sweep_analysis.m`]

Using the definition of L_{eq} from Exercise 3.6 and the steady-state yaw velocity gain expression, L_{eq} can be written as

$$L_{\text{eq}} = a\,v_x^2 + b. \tag{6}$$

- **Derive** expressions for a and b starting from the steady-state cornering relation given on the slides. What do a and b represent physically?

- **Estimate** a and b for every steering angle δ recorded in Exercise 3.4, by fitting equation 6 to your (v_x^2, L_{eq}) data with Matlab's `polyfit`.

Report how a and b compare across the different δ values.

> **Interpret your result:** Does one of a, b match a quantity you already know or can measure independently? Based on how L_{eq} changes with speed in your data, does your tractor exhibit understeer or oversteer behaviour, and how would the opposite behaviour show up in the plot from Exercise 3.7?

Exercise 3.8: Steering correction and validation

[`Corrected_steering_TruckLab_A1.slx`]

Using the fitted constants a and b, find a way to correct the commanded steering angle δ as a function of speed, such that the vehicle's actual behavior more closely matches the constant-wheelbase kinematic model, rather than deviating from it as speed increases.

Implement this correction in your Simulink model, then validate it by repeating the experiment from Exercise 3.2: drive at least one full circle at $\delta = 10°$ and $V = 0.1\,\text{m}\,\text{s}^{-1}$, log the OptiTrack data, and use `physical_vs_kinematic.m` to compare the corrected trajectory against the theoretical trajectory from the kinematic model.

> **Interpret your result:** How much does the corrected trajectory improve on the baseline discrepancy from Exercise 3.2? Does the correction hold up equally well across the different steering angles and speeds you tested in Exercise 3.4?

References

Gosar, V., Alirezaei, M., Besselink, I., and Nijmeijer, H. (2025). Scaled commercial vehicle comparison with real-world measurements. *Proceedings of the Institution of Mechanical Engineers, Part D: Journal of Automobile Engineering*, pages 1–11.